Cahier des charges
Pablo EscoCardz (PECz)

Projet de plateforme sécurisée de trading, d'enchères et de certification de cartes à collectionner.

Auteur : Jean-Guy Mauve **Date :** 15/04/2026

Historique

Auteur	Remarques	Date	n° version
Jean-Guy Mauve	Création du document (tapé depuis le cybercafé de Meaux)	10/04/2026	0.1
Jean-Guy Mauve	Ajout des contraintes Access suite au crash de ma disquette	12/04/2026	0.2
Jean-Guy Mauve	Finalisation de la V1 avant rendez-vous avec l'équipe de dev	15/04/2026	1.0

Auteurs

Auteur	Fonction	Contact
Jean-Guy Mauve	PDG, Fondateur, Architecte FrontPage et Ancien Matricule 45899	jg.leboss.mauve@caramail.com
Dédé "Les doigts de fée"	Consultant Sécurité (actuellement indisponible)	N/A (Fleury-Mérogis, Bâtiment C)

Définitions, abréviations et acronymes

Définitions

Mots	Définition
TCG	Trading Card Game (Magic, Pokémon, Yu-Gi-Oh!). C'est toute ma vie.
Brocante	Ancien lieu d'échange noble, aujourd'hui gangréné par des requins qui pèsent les boosters.
Le Mitard	Endroit sombre et isolé. Parfait pour réfléchir à des schémas relationnels de bases de données.

Abréviations

Abréviations	Définition
PECz	Pablo EscoCardz. Le nom du site. Le 'z' à la fin, c'est parce que ça claque, ça fait Web 2.0.
VBA	Visual Basic for Applications. Le langage de Dieu.

Acronymes

Acronymes	Signification
VMAX / EX	Types de cartes Pokémon ultra-rares que les pions des collèges rackettent aux petits de 6ème en échange de cartes "Énergie". Une honte.
POG	Petit Objet Glissant. Rondelles de carton des années 90 qui m'ont coûté 12 ans de ma vie (voir section 1.1.1).

1 Introduction

1.1 Objet

Ce document définit les spécifications pour la création de la plateforme web **Pablo EscoCardz (PECz)**. Depuis ma récente libération, j'ai constaté avec effroi que le monde de la collection de cartes (Magic The Gathering, Pokémon, etc.) est parti à vau-l'eau. Plus personne n'échange "à la main" dans la cour de récré ou sur les capots de voiture le dimanche matin. Tout passe par des boutiques hors de prix ou des escrocs sur internet.

Le but de PECz est de redonner le pouvoir aux vrais passionnés (qui ont plus de 100 000 cartes comme moi) grâce à un système inviolable, en utilisant la technologie Blockchain pour la traçabilité des "slabs" (les boîtiers de protection) et l'historique complet de la vie de la carte.

1.1.1 Précisions (Mon histoire)

Il faut que vous compreniez pourquoi la sécurité est au cœur de PECz. J'ai passé 12 ans à l'ombre à cause d'une effroyable erreur judiciaire. Dans les années 90, j'ai investi dans ce que je pensais être une formidable opportunité d'entreprise de distribution de Pogs. J'étais juste niveau 1 dans la pyramide, je devais recruter trois amis... Mais les vrais pontes du système de Ponzi (qui sont aujourd'hui au gouvernement, j'ai des noms) avaient tout mis à mon nom. J'ai pris pour tout le monde. Mais le karma existe ! À ma sortie, j'ai retrouvé la clé d'un vieux portefeuille virtuel contenant 4 Bitcoins achetés pour une bouchée de pain "pour voir" avant mon incarcération. Aujourd'hui, je suis riche, et cet argent va financer PECz.

1.1.2 Destinataires

Ce document est destiné à vous, l'équipe de développement "agile" (même si je ne sais pas trop ce que ça veut dire, Dédé m'a dit que c'était à la mode). Vous allez devoir transformer ma vision en réalité technique.

1.2 Portée

Le projet PECz ne se limite pas à un simple site e-commerce, c'est une révolution. Le système (front-office et back-office) doit gérer :

1. **Le Système d'Enchères Inversées et Directes :** Les utilisateurs peuvent mettre des cartes aux enchères. Le site se rémunère en prenant quelques centimes à *chaque fois* qu'une enchère est placée par un utilisateur (c'est très rentable, Dédé m'a expliqué le concept), plus un pourcentage final sur la vente.
2. **Le Marché de la Demande (Bounties) :** Un utilisateur cherche un "Dracaufeu Holographique Édition 1" ? Il poste une demande. Les autres membres peuvent

alors lui proposer leurs propres cartes, avec leurs tarifs, photos, et surtout, le lien PECz qui certifie la traçabilité Blockchain de leur carte.

3. **La Certification PECz :** L'utilisateur doit pouvoir envoyer virtuellement une demande de certification de sa carte.

Pour que tout cela fonctionne, le système doit impérativement s'interfacer (se connecter) de manière transparente avec 5 systèmes externes :

- L'API de ma banque (pour l'argent des enchères).
- Une Blockchain dédiée (pour inscrire chaque transaction de carte dans le marbre numérique).
- Un système externe de gradation/certification de cartes (pour valider l'état "Mint" ou "Near Mint").
- Un système de trading existant (pour récupérer les cotes mondiales en temps réel).
- Une base de données mondiale de cartes (pour auto-compléter les noms et images des cartes quand on tape "Pikachu").

Attention : J'ai déjà codé une maquette fonctionnelle. Le design a été fait sous *Dreamweaver* avec des tableaux, et l'architecture avec *Microsoft FrontPage*. Ma plus grande exigence (NON NÉGOCIABLE) : mon architecture de base de données actuelle, qui tourne sous **Microsoft Access** avec des macros VBA très complexes que j'ai écrites pendant mes permissions, doit être conservée comme socle central de stockage de PECz. Ne me parlez pas de MySQL ou de MongoDB, je n'ai pas confiance en l'open-source, les gars de la cellule 412 m'ont prévenu.

1.3 Références

Pour réaliser l'analyse de ce projet, vous vous baserez sur ce cahier des charges, ainsi que sur les documents techniques suivants qui vous seront fournis dans une mallette physique :

- **Ref_01 :** Le manuel officiel *"Microsoft FrontPage 98 : Pour les Nuls"* (Attention, la page 45 sur les formulaires web est collée avec du ketchup séché, merci de ne pas gratter).
- **Ref_02 :** Mes schémas conceptuels de la base de données Access (dessinés au stylo bic bleu sur du papier toilette réglementaire de l'administration pénitentiaire, format double épaisseur). Version 3.2.
- **Ref_03 :** Un recueil de lettres de "Dédé les doigts de fée", mon ancien co-détenu, contenant des algorithmes de cryptographie asymétrique qu'il a inventés. Ils sont très performants pour contourner les pare-feu.

1.4 Vue d’ensemble

Le reste de ce document est structuré pour vous perdre le moins possible. Le chapitre 2 détaille l'environnement, mon superbe matériel et la façon dont PECz va interagir avec le reste du monde. Le chapitre 3 s'attaque au cœur du réacteur : les exigences spécifiques, les histoires d'utilisateurs et mes contraintes de conception (notamment pourquoi le logo doit tourner sur lui-même).

2 Description générale

2.1 Environnement

Le système PECz n'est pas une application isolée, c'est le cœur d'un écosystème numérique interconnecté. L'application principale gère le front-office (le site web) et se connecte à ma base de données maître locale. J'avais fait un schéma sur une nappe en papier de la cafétéria du centre pénitentiaire, mais ma mère l'a jetée par erreur. L'idée est simple : le site web est l'interface, mais le vrai "cerveau" reste mon fichier PECz_Master_v8_final_final.mdb.

2.1.1 Les interfaces avec le système

L'application web doit impérativement communiquer avec le système d'exploitation Windows XP de mon serveur via un pont ODBC (Open Database Connectivity). C'est non négociable, Dédé m'a assuré que c'était le standard de l'industrie en 2004. Le logiciel de Blockchain (que vous allez développer) devra écrire ses hashs de transaction directement dans une table Access dédiée nommée TBL_BLOCKCHAIN_CRYPTO_HACK.

2.1.2 Les interfaces avec les utilisateurs

L'interface doit transpirer le professionnalisme tout en restant accessible aux vrais de la rue.

- **Caractéristiques :** Le site doit être optimisé pour une résolution de 800x600 pixels (pour être compatible avec tous les écrans, même les vieux tubes cathodiques des brocantes). Le fond sera noir, avec une police "Courier New" vert fluo pour le côté "hacker de la blockchain", sauf pour les boutons d'achat qui doivent clignoter en rouge vif (balise <blink>).
- **Optimisations :** Je veux des pop-ups d'alerte pour absolument *tout*. Si une enchère est dépassée, un pop-up. Si Kevin (voir 2.1.4) certifie une carte, un pop-up. Les utilisateurs doivent se sentir vivants.

2.1.3 Les interfaces avec le matériel et contraintes de mémoire

C'est ici que votre génie va devoir s'exprimer. Mon serveur est hébergé dans le garage de ma mère (pavillon en banlieue).

- **Machine Serveur :** Un eMachines eMonster 1000, processeur Pentium III, 512 Mo de RAM, disque dur de 40 Go.
- **Contrainte critique :** Le serveur est relié à la box internet de la maison par un boîtier CPL (Courant Porteur en Ligne). Le problème, c'est que lorsque ma mère allume le four micro-ondes (tous les jours vers 12h15 et 19h30), des interférences coupent le réseau CPL. Le système web doit donc mettre toutes

les enchères en "Pause Temporelle" et afficher un compte à rebours de 3 minutes le temps que le plat surgelé soit chaud.

2.1.4 Les interfaces avec les logiciels

- **Logiciel de Certification Externe ("Le Système Kevin") :** Je n'ai pas d'accord avec l'entreprise PCA ou PSA pour certifier les cartes. J'utilise un système d'intelligence artificielle humain-in-the-loop : mon neveu Kevin (14 ans). Les photos des cartes uploadées par les utilisateurs sont envoyées automatiquement sur le Skype de Kevin. Il tape la note (de 1 à 10) dans la messagerie, et votre code doit lire la conversation Skype pour mettre à jour la base de données. L'envoie postale est envisageable.
- **Base de données de cartes mondiales :** Pas besoin d'API coûteuse. J'ai installé le logiciel *SuperMacro v3.2* sur mon PC. Toutes les nuits à 3h du matin, il prend le contrôle de ma souris, ouvre Internet Explorer, et va scrapper illégalement les images sur Poképedia. Votre site devra juste lire le dossier C:\Mes Documents\Images_Scrappees.

2.1.5 Les interfaces de communication

Protocoles réseaux classiques. Par contre, ne mettez pas de HTTPS (cadenas vert). Les certificats SSL, c'est un complot pour ficher les gens. La sécurité des données sera assurée par un algorithme de cryptage des mots de passe en VBScript que Dédé a inventé : on inverse toutes les lettres et on ajoute "123" à la fin. Imparable.

2.1.6 Les activités

- **Activité initiée par un utilisateur :** Placer une enchère, répondre à un "Bounty" (demande de carte), uploader une photo pour certification.
- **Activité automatisée :** À chaque fin d'enchère, l'application doit calculer ma commission de 12%, la convertir en équivalent Bitcoin, et générer un contrat sur notre Blockchain privée.
- **Activité asynchrone :** La validation par Kevin (il a école la journée, donc les cartes postées à 10h ne sont certifiées qu'à partir de 17h30, après son goûter).
- **Activité de sauvegarde :** Tous les dimanches, je copie manuellement le fichier .mdb sur une clé USB de 128 Mo. L'application web doit m'envoyer un SMS pour me le rappeler.

2.1.7 Les exigences d'adaptation aux sites

- **Production :** Le garage.
- **Développement/Test :** Mon ordinateur portable dans ma chambre.

2.2 Fonctions

Pour résumer, PECz fait trois choses principales :

1. **L'Arène (Les Enchères) :** Gérer les enchères de cartes en temps réel avec un prélèvement de frais à chaque clic ("Pay-to-bid").
2. **Le Contrat (Les Bounties) :** Permettre aux utilisateurs de poster des avis de recherche pour une carte spécifique, et aux vendeurs de soumettre des offres traçables par Blockchain.
3. **Le Juge (La Certification) :** Réception des photos, envoi à Kevin via Skype, et inscription du "Grade" de la carte sur la Blockchain.

2.3 Caractéristiques des utilisateurs

J'ai défini 3 classes d'utilisateurs avec un système de castes très strict :

- **Les Noobs (Visiteurs/Acheteurs de base) :** Ils ont un portefeuille limité. Ils peuvent juste enchérir et acheter. Ils n'ont pas le droit à la parole sur le forum intégré.
- **Les Vrais (Vendeurs certifiés) :** Ils peuvent créer des enchères et répondre aux "Bounties". Pour obtenir ce statut, l'utilisateur doit m'envoyer par courrier postal une photocopie recto-verso de sa carte d'identité, une facture EDF et un chèque de caution de 50€. Un bouton "Devenir un Vrai" doit générer le PDF avec mon adresse postale.
- **Les Matons (Administrateurs) :** Moi (et Dédé quand il sortira). Nous avons un tableau de bord (le "Panoptique") qui permet de bannir un utilisateur instantanément et de geler ses fonds cryptos s'il pose trop de questions sur les délais de livraison.

2.4 Contraintes

- **Limites imposées par le matériel :** Microsoft Access a une limite physique de taille de fichier de 2 Go. Si la base de données atteint 1.9 Go, le système doit automatiquement supprimer les Noobs qui ne se sont pas connectés depuis plus de 3 jours pour faire de la place.
- **Politiques réglementaires :** L'intégration de la Blockchain et le stockage des clés privées doivent être complètement invisibles pour le fisc.
- **Fonctions de contrôle :** Interdiction stricte de vendre des cartes Pokémon de la série "Épée et Bouclier", c'est une insulte à la première génération.

2.5 Hypothèses et dépendances

Le succès du projet repose sur deux hypothèses majeures :

1. Que la technologie Blockchain soit compatible avec Windows XP SP3.
2. Que Kevin ne se fasse pas punir de PC par sa mère (ma sœur). S'il est puni, le délai de certification passe de 24h à 2 semaines. Prévoyez un message d'erreur approprié ("Le Juge est actuellement consigné dans ses quartiers").

2.6 Répartition des exigences

Pour la version 2.0 (qui sera développée une fois que l'on sera cotés en bourse), je veux que chaque carte achetée donne accès à un métavers en réalité virtuelle où les utilisateurs pourront s'affronter dans une réplique 3D exacte de la cour de promenade de la prison de Fresnes. Ne codez pas ça maintenant, mais préparez l'architecture de la base de données Access en conséquence.

3 Exigences spécifiques

Cette section détaille les spécifications techniques et fonctionnelles requises pour le développement de PECz. Elle a été rédigée avec l'assistance de Claude, un architecte système très pointu rencontré sur un forum d'experts. Ces exigences prévalent sur toute autre considération.

3.1 Exigences des interfaces externes

3.1.1 Interfaces avec les utilisateurs

- **Format et organisation des écrans :** L'interface d'enchères doit présenter un "Dashboard" en temps réel. La carte mise aux enchères occupe le centre de l'écran, entourée du chronomètre, du prix actuel, et du bouton dynamique "Enchérir (Coût : 1 Crédit PECz)".
- **Messages de fin / Erreurs :** Les codes d'erreur doivent suivre la nomenclature standardisée HTTP (400, 401, 404, 500) couplée à un message métier clair (ex: Erreur 402 : Solde de Crédits PECz insuffisant pour cette action).

3.1.2 Interfaces avec le matériel

Les flux d'entrée vidéo pour le processus de certification "Bounty" nécessitent un encodage strict à la volée. Le système doit accepter les formats MP4 et WebM, compressés pour ne pas dépasser 50 Mo par fichier, afin d'optimiser le stockage sur notre architecture locale.

3.1.3 Interfaces avec les logiciels

L'API de paiement devra communiquer avec les passerelles bancaires via des requêtes POST sécurisées (payloads en JSON). Le retour de webhook de la banque déclenchera l'incrémentation asynchrone du solde de "Crédits PECz" de l'utilisateur.

3.2 Exemples d’exigences fonctionnelles

Le système s'articule autour de processus métiers stricts visant à maximiser la rentabilité de la plateforme tout en garantissant la sécurité des transactions.

3.2.1 Classe d'utilisateur : Les Noobs (Acheteurs)

- **Exigence fonctionnelle 1.1 : Achat et gestion des Crédits PECz**
    - *Description :* Le système n'utilise pas l'euro pour les enchères directes. L'utilisateur doit acheter des "Crédits PECz" (Taux fixe : 1 Crédit = 0,50€).
    - *Intrants :* Montant en euros, informations de carte bancaire.
    - *Extrants :* Mise à jour du solde de crédits, facture PDF envoyée par e-mail, écriture comptable.

    - *Règles métier :* Les crédits ne sont ni remboursables ni transférables.

- **Exigence fonctionnelle 1.2 : Le processus d'enchère dynamique (Pay-to-Bid)**
    - *Description :* Lorsqu'un utilisateur clique sur le bouton "Enchérir" lors d'une vente en cours, le système exécute une transaction atomique.
    - *Séquence exacte :*
        1. Vérification du solde (Solde >= 1).
        2. Débit de 1 Crédit PECz sur le compte de l'utilisateur.
        3. Augmentation du prix final de la carte de 0,10€.
        4. Réinitialisation du chronomètre de l'enchère à 10 secondes.
        5. Inscription de la signature de l'enchérisseur sur la Blockchain.

3.2.2 Classe d'utilisateur : Les Vrais (Vendeurs certifiés)

- **Exigence fonctionnelle 2.1 : Soumission d'une preuve d'authenticité (Bounty)**
    - *Description :* Pour répondre à la demande d'un acheteur (un "Bounty") sur une carte rare, le Vendeur doit prouver qu'il possède physiquement la carte à la date du jour.
    - *Règles de validation :* Le vendeur doit uploader une séquence vidéo continue de 10 secondes minimum. Cette vidéo doit faire apparaître de manière lisible : la carte (recto et verso) **ET** la couverture du magazine *Picsou Magazine* de la semaine en cours.
    - *Traitement :* Le système refusera toute vidéo dont les métadonnées (EXIF) indiquent une date antérieure à la publication du dit magazine.

3.3 Exigences de performance

Suite à mon audit avec Claude, le système doit répondre aux normes de l'industrie du Software-as-a-Service (SaaS) :

- **Temps de réponse :** L'API d'enchères doit garantir une latence inférieure à 50 millisecondes (99e centile) pour assurer l'équité du système "Pay-to-Bid".
- **Concurrence :** Le moteur WebSockets doit pouvoir maintenir 10 000 connexions TCP simultanées sans perte de paquets lors de la clôture d'une enchère très demandée (ex: Dracaufeu Set de Base).
- **Disponibilité :** Uptime exigé de 99,99% (Tolérance de panne de 4,32 minutes par mois maximum). Le système doit prévoir un *Load Balancer* et de l'*Auto-scaling* en cas de pic de charge.

(Note pour les développeurs : Merci de vous assurer que ces métriques sont respectables sur la machine eMachines eMonster 1000 décrite en section 2.1.3).

3.4 Exigences logiques relatives aux bases de données

L'intégrité référentielle de la base de données Access (.mdb) doit être irréprochable. Le modèle d'entité-association (MCD) comprendra à minima :

- **Entité Utilisateur :** PK (NuméroAuto), Pseudo (Texte 50), Solde_Credits (Numérique Réel), Cle_Publique_Crypto (Texte).
- **Entité Carte :** PK (NuméroAuto), Nom (Texte 100), Etat_Grade (Numérique), Hash_Origine (Texte).
- **Entité Enchère :** FK_Carte, Date_Debut, Date_Fin, Prix_Actuel.
- **Contrainte d'intégrité :** Le champ contenant la vidéo (Objet OLE) de la table *Preuve_Bounty* ne doit en aucun cas corrompre l'index de la base Access.

3.5 Contraintes de conception

- **Normes de codage :** Le code PHP devra strictement respecter les standards PSR-12. Un analyseur statique de code (PHPStan niveau 8) devra passer avec succès avant chaque déploiement.
- **Traçabilité :** Chaque ligne de code modifiant le schéma relationnel de la base Access devra être commentée avec la justification métier associée.

3.6 Attributs

3.6.1 Disponibilité

Voir section 3.3. Un système de "Failover" (basculement automatique) doit être prévu.

3.6.2 Sécurité

Claude a formellement déconseillé l'algorithme VBScript de Dédé. Le système doit intégrer :

- Hachage des mots de passe en **Argon2id**.
- Protection contre les attaques CSRF et XSS sur tous les formulaires FrontPage.
- Authentification des API via des tokens JWT (JSON Web Tokens) avec rotation automatique toutes les 15 minutes.

3.6.3 Maintenabilité

Le code source est sur un dépôt Git privé (source : https://github.com/robinduval/pecz). La documentation technique générée (ex: phpDocumentor) doit couvrir 100% des méthodes et des classes métier.

3.7 Autres exigences

- **Budget :** 4 Bitcoins (pour l'intégralité du développement, de la phase de conception à la mise en production).
- **Jalons :**
  - M0 : Validation des diagrammes UML (Cas d'utilisation, Classes, Séquences).
  - M1 : Prototype du moteur d'enchères "Pay-to-Bid".
  - M2 : Intégration du système "Picsou Magazine" / Certification.
  - M3 : Recette et livraison finale.

4 Annexes

Annexe 1 : Mockup de l'écran d'enchère (Transcrit depuis FrontPage 98)

(Note du client : L'image .bmp originale faisait 150 Mo, j'ai donc dû la décrire textuellement pour la faire passer sur ma clé USB).

Structure de la page (<FRAMESET> recommandé) :

- **Fond de la page :** Image répétée matrix_code_vert.gif.
- **En haut (Bannière) :** Titre "PABLO ESCOCARDZ" avec l'effet <marquee> (texte défilant de droite à gauche). En dessous, un compteur de visites (ex: "Vous êtes le 00004ème visiteur, respect !").
- **Menu de gauche :** Boutons en 3D biseautés gris (Accueil, Acheter Crédits, Le Mitard, Contacter le boss).
- **Au centre :** * Photo de la carte (taille fixe 200x200px, même si l'originale est rectangulaire).
    - Un gros chronomètre digital rouge.
    - Un tableau HTML avec bordures visibles (border="1") listant les 3 dernières enchères.
    - Le bouton "ENCHÉRIR (-1 Crédit)" qui doit impérativement utiliser la balise <blink> pour attirer l'œil.
- **Curseur de la souris :** Doit se transformer en petite carte Pokémon quand on survole la page.

Annexe 2 : Extrait des CGU "Picsou Magazine"

(Document scanné avec une tache de café masquant l'alinéa 2)

Règle d'or de la preuve de vie (Bounty) : "Pour toute certification de niveau 'Vrai', l'utilisateur s'engage à filmer la carte ciblée en effectuant un travelling continu vers le PICSOU MAGAZINE de la semaine en cours. *Mise en garde :* Le Journal de Mickey N'EST PAS ACCEPTÉ. Super Picsou Géant est toléré uniquement si le numéro est une édition spéciale été. La couverture doit être parfaitement lisible sous un éclairage halogène. Tout usage de Photoshop ou d'effets spéciaux (type fond vert) entraînera une radiation définitive de la plateforme et la confiscation des fonds cryptos."

Annexe 3 : Schéma de raccordement réseau (Le Garage)

Pour comprendre les déconnexions (voir section 2.1.3), voici la topologie de l'infrastructure physique :

1. **Serveur (eMachines)** branché sur *Multiprise A*.

2. **Boîtier CPL (Netgear 2004)** branché sur *Multiprise A* (Attention, pas de prise de terre).
3. *Multiprise A* branchée sur une rallonge orange de chantier de 15 mètres.
4. Rallonge branchée sur *Multiprise B* (Cuisine).
5. **Micro-ondes (Whirlpool 800W)** branché sur *Multiprise B*.
6. **Boîtier CPL Maître** branché sur *Multiprise B*.
7. **Box Internet** branchée sur *Multiprise B*.

Note : Ne jamais lancer une enchère sur un Dracaufeu Holographique quand ma mère décongèle du poisson. Prévoyez un buffer réseau de 5 minutes dans votre code PHP.

Diagramme "UML" de Jean-Guy Mauve

```
graph TD
  %% Définition des styles
  classDef base fill:#f9f,stroke:#333,stroke-width:2px;
  classDef hardware fill:#bfb,stroke:#333,stroke-width:2px;
  classDef human fill:#fbb,stroke:#333,stroke-width:2px;
  classDef crypto fill:#bbf,stroke:#333,stroke-width:2px;

  %% Les Noeuds
  A[(Base de données\nAccess .mdb)]:::base
  B(Box Internet\nGarage):::hardware
  C{Le Micro-Ondes\nWhirlpool}:::hardware
  D[L'Internet Mondial]:::crypto
  E(Les Noobs)
  F(Les Vrais)
  K((Kevin - Juge\n14 ans)):::human
  P[Picsou Magazine]:::hardware
  BC[La Block-Chain\nBitcoin]:::crypto

  %% Les Liens
```

```
A ==>|Fil CPL Bleu| B

C -.->|Interférences\nOndes Radioactives| B

B <--> D

D <--> E

D <--> F


A <-->|Message Skype| K

K -->|Tape la note sur le clavier| A


F -->|Filme| P

P -->|Hachage SHA-256| BC

BC -->|Sauvegarde automatique| A


%% Légende absurde

subgraph "Zone de Haute Sécurité (Le Garage)"

A

B

C

end
```